

Informe #2

Definición de la Metodología del Proyecto



Integrantes:
Owuen Yagual Panimboza
Stiven Rivera Palma

12 de Junio, 2026

Contents

1	Introduction	2
1.1	Descripción del problema	2
1.2	Justificación del problema	3
1.3	Objetivos	3
1.3.1	Objetivo principal	3
1.3.2	Objetivos específicos	3
1.4	Módulos del proyecto	3
1.4.1	Módulo 1 – Captura y Preprocesamiento de Imagen	4
1.4.2	Módulo 2 – Detección de Pose con MediaPipe	4
1.4.3	Módulo 3 – Cálculo de Ángulos Articulares	5
1.4.4	Módulo 4 – Simulación del Brazo Robótico Virtual	6
1.4.5	Módulo 5 – Visualización y Métricas en Tiempo Real	7
1.5	Soluciones existentes	7
2	Metodología	9
2.1	Análisis de los Datos	9
2.1.1	Datasets de Referencia	9
2.1.2	Análisis de Datos por Módulo	9
2.1.3	Análisis de Confiabilidad y Compatibilidad	10
2.2	Alcance del Proyecto	11
2.2.1	Requerimientos	11
2.2.2	Recursos a Utilizar	12
2.2.3	Diseño Conceptual	12
2.2.4	Bocetos y Flujos de Pantalla	14
2.3	Plan de Implementación del Proyecto	15
2.3.1	Gestión de Riesgos	16
	References	17

Chapter 1

Introduction

1.1 Descripción del problema

La captura de movimiento markerless (sin marcadores físicos) representa una línea activa de investigación en visión artificial con aplicaciones en robótica, rehabilitación médica, interfaces humano-máquina e interacción natural. [1]

Los enfoques tradicionales basados en sensores físicos, trajes de captura óptica o cámaras de profundidad resultan costosos e intrusivos. El presente proyecto propone un sistema basado exclusivamente en una cámara convencional que, procesando el video en tiempo real, detecte la pose del brazo humano, calcule los ángulos articulares de hombro, codo y muñeca, y transfiera esos valores a la simulación visual de un brazo robótico virtual de tres grados de libertad (GDL). [2]

La solución adopta MediaPipe Pose de Google, un framework de código abierto que proporciona detección de 33 landmarks corporales en tiempo real sin necesidad de GPU, combinado con OpenCV para la adquisición y el procesamiento de imagen. [3] [4]

Diagrama de Bloques General del Sistema

La figura 1. representa el flujo de datos global del proyecto, descompuesto en sus cinco módulos funcionales principales.

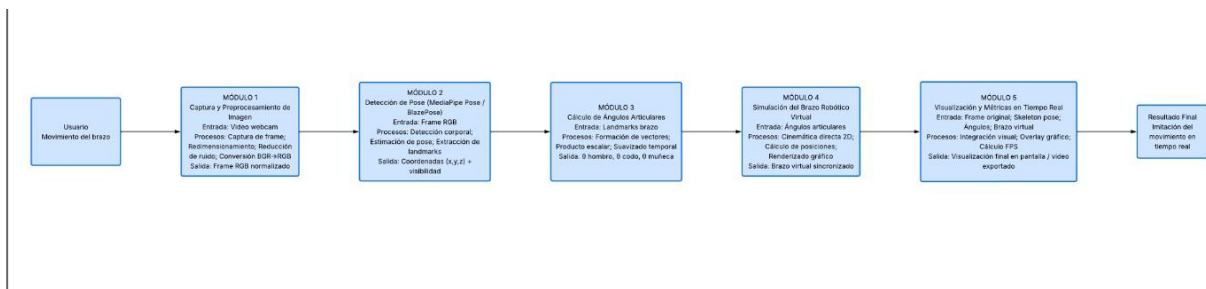


Figure 1. – Diagrama de bloques del sistema de imitación de movimientos con MediaPipe Pose.

Cada bloque representa un módulo funcional independiente con entradas y salidas definidas. El video de la cámara entra al sistema por M1, los landmarks son extraídos en M2, los ángulos se calculan en M3, el brazo robótico es simulado en M4 y todo se integra visualmente en M5. [3] [5]

1.2 Justificación del problema

MediaPipe fue presentado originalmente como un framework modular para construir pipelines de percepción reproducibles entre plataformas. Su componente BlazePose amplió esta capacidad al dominio de la estimación de pose corporal en tiempo real, logrando latencias menores a 30 ms en CPU mediante un pipeline de dos etapas: detector de persona (BlazePose Detector) y estimador de pose (BlazePose Estimator) que extrae 33 puntos clave. [3] [4]

Trabajos previos como los de Pauzi et al. (2021) demostraron la efectividad de BlazePose para estimar movimientos corporales con una diferencia de precisión dentro del 10% respecto a sistemas IMU de referencia. En el dominio de la teleoperación robótica, Altayeb (2023) implementó un sistema de replicación de gestos de mano en un brazo robótico usando MediaPipe Hands, validando la viabilidad de este enfoque para control de actuadores virtuales. [1] [6]

La justificación económica radica en el bajo costo de implementación: el sistema requiere únicamente una webcam USB estándar (< \$20 USD), sin sensores de profundidad ni hardware especializado. El stack de software es completamente open-source (Python, OpenCV, MediaPipe, NumPy).

1.3 Objetivos

1.3.1 Objetivo principal

Desarrollar un sistema de visión artificial que, a partir de video capturado en tiempo real por una cámara convencional, detecte los movimientos del brazo humano utilizando MediaPipe Pose [3] [4], calcule los ángulos de las articulaciones hombro, codo y muñeca, y los transfiera a la simulación visual sincronizada de un brazo robótico virtual de 3 GDL.

1.3.2 Objetivos específicos

1. Implementar un módulo de captura y preprocesamiento de video en tiempo real usando OpenCV [5] con soporte para normalización de resolución (640×480) y conversión de espacio de color BGR→RGB.
2. Integrar el modelo MediaPipe Pose [3] para la detección y extracción de los 33 landmarks corporales, con énfasis en los puntos clave del brazo (hombros: landmarks 11,12; codos: 13,14; muñecas: 15,16).
3. Desarrollar un módulo de cálculo de ángulos articulares mediante álgebra vectorial (producto escalar y norma de vectores) para los tres GDL del brazo: hombro (elevación), codo (flexión) y muñeca (flexión), con filtrado de media móvil de 5 frames.
4. Implementar la simulación 2D del brazo robótico virtual mediante cinemática directa, renderizando en tiempo real el modelo sobre el frame de video usando primitivas OpenCV (cv2.line, cv2.circle) [5].
5. Construir un módulo de visualización integrada que muestre el skeleton de MediaPipe [4], los ángulos articulares calculados, el brazo robótico simulado y métricas de rendimiento (FPS).
6. Evaluar el desempeño del sistema con métricas de latencia (FPS objetivo: ≥ 20 FPS), precisión angular y estabilidad del sistema ante variaciones de iluminación, usando como referencia el benchmark del dataset COCO Keypoints [7].

1.4 Módulos del proyecto

El sistema se descompone en cinco módulos funcionales interconectados. La arquitectura modular facilita el desarrollo y prueba independiente de cada componente en el marco SCRUM adoptado. Las tecnologías principales son OpenCV [5] para procesamiento de imagen y MediaPipe Pose [3, 7] para estimación de pose.

1.4.1 Módulo 1 – Captura y Preprocesamiento de Imagen

Objetivo

Adquirir el flujo de video en tiempo real desde la cámara y preparar cada frame para su procesamiento posterior mediante normalización de resolución, conversión de espacio de color y reducción de ruido.

	Descripción	Formato / Especificación
Entrada	Flujo de video desde webcam USB o cámara integrada	Resolución variable (640×480 – 1920×1080), 30 FPS
Salida	Frame de imagen normalizado en espacio RGB	NumPy array RGB, resolución fija 640×480, píxeles en [0, 255]

Table 1. – Entradas y salidas del Módulo 1.

Entradas y Salidas

Herramientas Tecnológicas

- **Software:** Python 3.10+ como lenguaje de implementación.
- **Librería principal:** OpenCV (cv2 v4.8+) [5] — captura de video (cv2.VideoCapture), redimensionado (cv2.resize) y conversión de espacio de color (cv2.cvtColor).
- **Librería de soporte:** NumPy (v1.24+) para manejo y operaciones sobre arrays de imagen.

Técnicas de Visión Computacional

- **Realzado y restauración:** aplicación opcional de filtro gaussiano (cv2.GaussianBlur) para atenuación de ruido de alta frecuencia antes de la inferencia de MediaPipe [8].
- Redimensionado con interpolación bicúbica (cv2.INTER_CUBIC) para preservar calidad de bordes al escalar el frame.
- Conversión de espacio de color BGR→RGB requerida por el modelo BlazePose de MediaPipe.

1.4.2 Módulo 2 – Detección de Pose con MediaPipe

Objetivo

Detectar los 33 landmarks del cuerpo humano en cada frame de video utilizando el modelo MediaPipe Pose (BlazePose), con énfasis en los puntos clave del brazo: hombros (landmarks 11, 12), codos (13, 14) y muñecas (15, 16) [3].

Entradas y Salidas

Herramientas Tecnológicas

- **Modelo de IA preentrenado:** MediaPipe Pose (BlazePose) v0.10+ [3, 4] — modelo de estimación de pose de 33 landmarks entrenado sobre el dataset COCO Keypoints. No requiere GPU.

	Descripción	Formato / Especificación
Entrada	Frame RGB preprocesado (salida de M1)	NumPy array RGB 640×480
Salida	33 landmarks corporales con visibilidad	NormalizedLandmarkList: coord. (x, y, z) $\in [0, 1]$ + visibilidad $\in [0, 1]$ por punto

Table 2. – Entradas y salidas del Módulo 2.

- **Framework:** MediaPipe (Google) [4] — framework de construcción de pipelines de percepción con soporte cross-platform.
- **Librería de soporte:** NumPy para extracción y transformación de las coordenadas de landmarks al espacio de píxeles.

Técnicas de Visión Computacional

- **Pipeline BlazePose de dos etapas:**
 1. Detector de persona que delimita el bounding box de la región de interés.
 2. Estimador de pose que infiere los 33 landmarks con alta precisión dentro de esa región.
- **Modo de video** (`static_image_mode=False`): el modelo aprovecha la coherencia temporal entre frames consecutivos para acelerar la inferencia y estabilizar los landmarks.
- **Extracción de landmarks de interés:** se seleccionan los índices 11–16 (hombros, codos, muñecas) de los 33 puntos detectados para alimentar el módulo de cálculo angular.

1.4.3 Módulo 3 – Cálculo de Ángulos Articulares

Objetivo

Calcular los ángulos de flexión/extensión en las articulaciones hombro, codo y muñeca a partir de las coordenadas 2D de los landmarks, usando la definición vectorial del ángulo entre dos segmentos articulares.

Entradas y Salidas

	Descripción	Formato / Especificación
Entrada	Coordenadas 2D de hombro, codo y muñeca (salida de M2)	Pares (x, y) en píxeles para cada articulación (landmarks 11–16)
Salida	Tripleta de ángulos articulares suavizados	$\theta_{\text{hombro}}, \theta_{\text{codo}}, \theta_{\text{muñeca}}$ en grados $[0, 180]$ (float64)

Table 3. – Entradas y salidas del Módulo 3.

Herramientas Tecnológicas

- **Librería principal:** NumPy (v1.24+) para cálculo del producto escalar (`np.dot`), norma de vectores (`np.linalg.norm`) y función arccos (`np.arccos`) necesarios en la fórmula angular.
- **Librería de soporte:** Matplotlib (v3.7+) para graficar la evolución temporal de los ángulos articulares durante las pruebas y análisis de resultados.

Técnicas de Visión Computacional y Geometría

- **Cálculo de ángulos mediante producto escalar:** [8]

$$\theta = \arccos\left(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{|\mathbf{v}_1| |\mathbf{v}_2|}\right) \quad (1.1)$$

donde $\mathbf{v}_1$ y $\mathbf{v}_2$ son los vectores que forman cada par de segmentos en la articulación.

- **Filtrado temporal con media móvil** de ventana $w = 5$ frames para suavizar fluctuaciones por ruido de detección de BlazePose y evitar movimientos bruscos en la simulación del brazo robótico.
- **Cálculo de vectores entre articulaciones:** diferencia de coordenadas entre el landmark proximal y el distal para construir cada vector de segmento corporal.

1.4.4 Módulo 4 – Simulación del Brazo Robótico Virtual

Objetivo

Representar y animar en pantalla un brazo robótico de 3 GDL cuyas posiciones articulares se corresponden directamente con los ángulos calculados por M3, logrando una imitación visual sincronizada en tiempo real [6].

Entradas y Salidas

	Descripción	Formato / Especificación
Entrada	Ángulos articulares suavizados (salida de M3)	$\theta_{\text{hombro}}, \theta_{\text{codo}}, \theta_{\text{muñeca}}$ en grados (float64)
Salida	Frame con overlay del brazo robótico virtual	NumPy array BGR con eslabones y articulaciones renderizados en tiempo real

Table 4. – Entradas y salidas del Módulo 4.

Herramientas Tecnológicas

- **Librería principal:** OpenCV (cv2) [5] para renderizado de primitivas gráficas: cv2.line (eslabones), cv2.circle (articulaciones), cv2.putText (ángulos).
- **Librería de soporte:** NumPy para el cálculo de coordenadas cartesianas de cada articulación mediante cinemática directa 2D.

Técnicas de Visión Computacional y Robótica

- **Cinemática directa 2D:** a partir de los ángulos articulares y las longitudes de eslabón predefinidas ($L_{\text{hombro}}, L_{\text{antebrazo}}, L_{\text{mano}}$), se calculan las posiciones cartesianas (x, y) de cada junta del brazo virtual mediante relaciones trigonométricas acumulativas:

$$\begin{aligned} x_1 &= x_0 + L_{\text{hombro}} \cos(\theta_{\text{hombro}}) \\ y_1 &= y_0 - L_{\text{hombro}} \sin(\theta_{\text{hombro}}) \\ x_2 &= x_1 + L_{\text{antebrazo}} \cos(\theta_{\text{hombro}} + \theta_{\text{codo}}) \\ y_2 &= y_1 - L_{\text{antebrazo}} \sin(\theta_{\text{hombro}} + \theta_{\text{codo}}) \end{aligned} \quad (1.2)$$

- **Renderizado de overlay:** el brazo robótico se dibuja sobre el frame de video como una capa superpuesta, permitiendo la comparación visual directa entre el movimiento humano y la imitación del brazo virtual.

1.4.5 Módulo 5 – Visualización y Métricas en Tiempo Real

Objetivo

Integrar y mostrar en una ventana unificada el frame original anotado con el skeleton de MediaPipe, los ángulos calculados, el brazo robótico virtual y las métricas de rendimiento del sistema (FPS).

Entradas y Salidas

	Descripción	Formato / Especificación
Entrada	Frame original + landmarks + ángulos + overlay del brazo robótico	Salidas de M1, M2, M3 y M4 en cada iteración del bucle principal
Salida	Frame anotado final visualizado en tiempo real	Ventana OpenCV (cv2.imshow) con overlay completo; exportación opcional a MP4 (cv2.VideoWriter)

Table 5. – Entradas y salidas del Módulo 5.

Herramientas Tecnológicas

- **Librería principal:** OpenCV (cv2) [5] — visualización en ventana interactiva (cv2.imshow, cv2.waitKey) y exportación de video (cv2.VideoWriter).
- **MediaPipe Drawing Utils** [4] — dibujo automático del skeleton de 33 landmarks y conexiones corporales sobre el frame.
- Matplotlib (v3.7+) — generación de gráficas de evolución temporal de ángulos articulares y métricas de FPS para análisis de resultados.
- scikit-learn (v1.3+) — cálculo de métricas estadísticas de evaluación: error medio angular (MAE), correlación de Pearson y análisis de estabilidad del sistema.

Técnicas de Visión Computacional

- **Cálculo de FPS en tiempo real:** medición del tiempo de procesamiento por frame con timestamps de Python (time.time()) para monitoreo continuo del rendimiento del sistema.
- **Visualización compuesta:** superposición de múltiples capas (frame original, skeleton MediaPipe, overlay robótico, texto de métricas) mediante operaciones de blending de OpenCV.

1.5 Soluciones existentes

Hoy en día, replicar los movimientos del cuerpo humano mediante visión por computadora es un campo clave para disciplinas como la robótica, la interacción persona-ordenador, la medicina de rehabilitación y la automatización. En la literatura actual encontramos diferentes estrategias para resolver esto, pero casi todas comparten el mismo fin: capturar lo que hace un usuario, calcular dónde están sus articulaciones en el espacio y trasladar esa información a un robot o a un entorno digital.

Una de las líneas de investigación con más fuerza es la teleoperación por imitación. Aquí el reto principal no es solo registrar el movimiento de los brazos o piernas del usuario, sino ver cómo adaptar esos datos a la estructura de un robot, cuyas articulaciones y límites físicos suelen ser muy diferentes a los de un humano.

Por ejemplo, Zhang y su equipo diseñaron un sistema para conectar los movimientos de un brazo humano con un robot industrial. Lo interesante de su propuesta es que crearon un modelo cinemático capaz de traducir las trayectorias en tiempo real; el robot lograba replicar la orientación del brazo incluso teniendo una anatomía completamente distinta [9]. Esto demostró que la compatibilidad física no es una barrera insalvable. En una línea parecida, Xuan y Tang apostaron por la simplicidad del hardware al usar una sola cámara web común (óptica monocular) para controlar un brazo robótico [10]. Su trabajo dejó claro que no hacen falta sensores pegados al cuerpo ni trajes especiales si se cuenta con un buen procesamiento visual.

Por otro lado, cuando se trabaja con robots humanoides, la complejidad aumenta. En otro estudio, Han et al. se enfocaron en este problema desarrollando un método para rastrear específicamente el codo y la muñeca del operario. Su sistema ajusta esas coordenadas sobre la marcha para que el robot aprenda tareas de manipulación sin perder el equilibrio ni la fluidez [11]. Más recientemente, un estudio de la Universidad de los Andes (2026) propuso usar cámaras RGB-D (que miden profundidad) junto con algoritmos de estimación de pose. Su investigación confirmó que la visión artificial actual es lo suficientemente madura como para interactuar con gemelos virtuales y robots físicos con una precisión bastante aceptable, dejando atrás los incómodos sensores portables [12].

Al revisar todas estas soluciones, salta a la vista que casi todos los autores siguen una misma receta: capturan el video, detectan los puntos clave del cuerpo, analizan la geometría del movimiento y lo envían al dispositivo. Sin embargo, hay un problema: casi toda la investigación actual se empeña en usar robots físicos caros o equipos muy específicos. Es justo ahí donde este proyecto quiere marcar la diferencia. En lugar de lidiar con hardware costoso, proponemos crear una simulación con un brazo robótico virtual. De esta forma, reducimos los costos y la complejidad técnica a cero, pero manteniendo el foco en lo importante: experimentar y validar algoritmos de visión artificial y procesamiento de imágenes en tiempo real.

Chapter 2

Metodología

2.1 Análisis de los Datos

El sistema opera principalmente sobre datos de video RGB en tiempo real capturados por una cámara convencional. A continuación se presenta el análisis de los datasets de referencia utilizados para la evaluación del desempeño, así como las características de los datos que maneja cada módulo del proyecto.

2.1.1 Datasets de Referencia

Para la evaluación y benchmarking del sistema se utilizarán datasets públicos consolidados en el campo de estimación de pose humana. La elección de estos conjuntos de datos se justifica por su amplia adopción en la literatura científica y su compatibilidad con el modelo MediaPipe Pose. [3, 7]

Dataset	Tipo	Imágenes / Videos	Landmarks	Uso en el proyecto
COCO Keypoints	Imágenes estáticas RGB	328K imágenes, 2.5M instancias	17 keypoints	Benchmark de referencia para evaluar precisión de MediaPipe Pose
LSP	Imágenes RGB de deporte	2000 imágenes	14 keypoints	Validación del cálculo angular en poses con alta variabilidad

Table 1. – Datasets de referencia para evaluación del sistema.

2.1.2 Análisis de Datos por Módulo

Cada módulo del sistema procesa un tipo de dato diferente. La siguiente tabla detalla las características de los datos de entrada, las propiedades relevantes y el preprocesamiento necesario en cada etapa del pipeline.

Módulo	Tipo de dato de entrada	Características clave	Preprocesamiento necesario
M1. Captura	Frame BGR de video (uint8, 3 canales)	Resolución variable; posible ruido de sensor; iluminación no controlada	Redimensionado a 640×480; conversión BGR→RGB; filtro gaussiano opcional
M2. MediaPipe Pose	Frame RGB normalizado	Sujeto visible; cuerpo parcialmente ocluido hasta aproximadamente un 30%	Reflejo horizontal opcional (modo selfie); ajuste de min_detection_confidence ≥ 0.7 ; extracción de landmarks 11, 13, 15, 17, 19, 21 (hombro, codo, muñeca, nudillo índice, punta índice y punta pulgar derechos)
M3. Ángulos	Landmarks 2D (float32, $x, y \in [0, 1]$)	Coordenadas normalizadas de 7 landmarks; fluctuaciones de ± 2 a ± 5 px por ruido de BlazePose; distancia euclidiana entre landmarks 19 y 21 para apertura del gripper	Desnormalización a píxeles; filtro media móvil $w = 5$; verificación de visibilidad > 0.5
M4. Brazo virtual	Tripleta de ángulos (float64, grados)	Valores continuos $[0, 180]$; posibles discontinuidades en límites angulares	Clipping a rango físico válido por articulación; interpolación lineal para suavizado extra
M5. Visualización	Frame BGR + overlays compuestos	Alta variabilidad según iluminación y fondo	Sin preprocesamiento adicional; medición de FPS por diferencia de timestamps

Table 2. – Análisis de datos de entrada, características y preprocesamiento por módulo.

2.1.3 Análisis de Confiabilidad y Compatibilidad

Se analiza la confiabilidad de las principales soluciones existentes y su compatibilidad con los requisitos del proyecto. MediaPipe Pose (BlazePose) ha sido validado con una diferencia de precisión inferior al 10% respecto a sistemas IMU de referencia en estimación de movimientos corporales. [1]

En términos de compatibilidad tecnológica, el stack seleccionado (Python + OpenCV + MediaPipe) presenta las siguientes garantías: (a) compatibilidad multiplataforma (Windows, Linux, macOS) sin modificación del código; (b) ausencia de dependencias de GPU, lo que elimina problemas de compatibilidad de drivers; (c) todas las librerías son de código abierto con licencias Apache 2.0 o BSD, sin restricciones de uso académico; y (d) versiones estables con soporte activo a la fecha del proyecto (2026). [3–5]

- **MediaPipe Pose v0.10+:** estable desde 2022, soportado en Python 3.8–3.12, compatible con OpenCV 4.x. Latencia de inferencia < 30 ms en CPU. [3]
- **OpenCV v4.8+:** librería de visión computacional más usada globalmente, con más de 20 años de desarrollo activo y amplia documentación. [5]
- **NumPy v1.24+:** librería estándar de álgebra numérica en Python, prerequisite de todas las demás librerías del stack.

2.2 Alcance del Proyecto

El alcance del proyecto queda delimitado por los siguientes criterios: el sistema procesa video de una sola persona visible en cámara (single-person pose estimation), en entornos con iluminación entre 50 y 500 lux, con el sujeto a una distancia de 1–3 metros de la cámara. El brazo robótico virtual es una simulación 2D renderizada en pantalla y no controla hardware físico. [2]

2.2.1 Requerimientos

Requerimientos Funcionales

ID	Requerimiento	Prioridad	Módulo(s)
RF-01	El sistema debe capturar video en tiempo real desde una cámara RGB a mínimo 30 FPS	Alta	M1
RF-02	El sistema debe detectar la pose del cuerpo humano usando MediaPipe Pose con una confianza mínima de 0.7, extrayendo únicamente los landmarks 11, 13, 15, 17, 19 y 21 correspondientes al hombro, codo, muñeca, nudillo, punta del índice y punta del pulgar derechos	Alta	M2, M3
RF-03	El sistema debe calcular los ángulos de hombro, codo y muñeca con un error máximo de $\pm 5^\circ$	Alta	M3
RF-04	El sistema debe aplicar filtro de media móvil de ventana $w = 5$ para suavizar los ángulos calculados	Media	M3
RF-05	El sistema debe renderizar un brazo robótico virtual de 3 GDL sincronizado con el movimiento en tiempo real	Alta	M4
RF-06	El sistema debe mostrar los ángulos articulares y las métricas de FPS superpuestas en el frame de video	Media	M5
RF-07	El sistema debe exportar el video procesado a un archivo MP4 cuando el usuario lo requiera	Baja	M5
RF-08	El sistema debe procesar al menos 20 FPS en hardware de CPU estándar (sin GPU)	Alta	M1, M5
RF-09	El sistema debe calcular la apertura del gripper como la distancia euclidiana normalizada entre la punta del índice (landmark 19) y la punta del pulgar (landmark 21), mapeada al rango [0%, 100%]	Alta	M3, M4

Table 3. – Requerimientos funcionales del sistema.

Requerimientos No Funcionales

ID	Requerimiento	Criterio de aceptación
RNF-01	Rendimiento: ≥ 20 FPS en CPU Intel Core i5 con 8 GB RAM	FPS promedio medido sobre 500 frames consecutivos ≥ 20
RNF-02	Precisión angular: error medio (MAE) $\leq 5^\circ$ respecto a medición de referencia	MAE calculado sobre 100 poses con transportador digital de referencia
RNF-03	Robustez: detección estable con variaciones de iluminación entre 50 y 500 lux	Tasa de detección exitosa $\geq 90\%$ en el rango de iluminación especificado
RNF-04	Portabilidad: ejecución en Windows 10+ y Ubuntu 20.04+ sin modificaciones	Instalación y ejecución exitosa en las tres plataformas con el mismo código fuente
RNF-05	Usabilidad: tiempo de configuración inicial ≤ 10 minutos	Seguir la guía de instalación del README y completarla en el tiempo estipulado
RNF-06	Mantenibilidad: código estructurado en módulos independientes con docstrings en cada función	Cobertura de docstrings $\geq 80\%$ verificada con pydocstyle por ejemplo.

Table 4. – Requerimientos no funcionales del sistema.

2.2.2 Recursos a Utilizar

Hardware

Recurso	Especificación mínima	Justificación
Computador Laptop	Intel Core i5 (8ª gen+) o AMD Ryzen 5; 8 GB RAM; Windows 10 / Ubuntu 20.04	MediaPipe Pose opera en CPU sin GPU; la especificación mínima garantiza ≥ 20 FPS
Cámara web (webcam)	Resolución mínima 640×480 @ 30 FPS; interfaz USB 2.0 o integrada	Compatible con cv2.VideoCapture(0); cualquier cámara reconocida por el SO
Espacio en disco	500 MB libres para entorno virtual y dependencias	Python + OpenCV + MediaPipe + dependencias con tamaño aproximado de 450 MB
Conexión a internet	Necesaria solo para instalación inicial de paquetes pip	Requerida en Sprint 2 para descargar modelos BlazePose (mas o menos 25 MB)

Table 5. – Recursos de hardware requeridos.

Software y Librerías

2.2.3 Diseño Conceptual

El diseño conceptual del sistema se articula en torno a un pipeline de procesamiento en tiempo real compuesto por los cinco módulos definidos en el Sprint #1. El flujo completo se ejecuta en un bucle principal a la frecuencia de la cámara (30 FPS objetivo). El tiempo total de procesamiento por frame debe ser inferior a 50 ms para garantizar el requisito de ≥ 20 FPS. [3, 5]

Librería / Herramienta	Versión	Licencia	Rol en el proyecto
Python	3.10+	PSF	Lenguaje principal de implementación
OpenCV (cv2)	4.8+	Apache 2.0	Captura, preprocesamiento, renderizado y visualización (M1, M4, M5) [5]
MediaPipe	0.10+	Apache 2.0	Detección de pose BlazePose; extracción de landmarks 11, 13, 15, 17, 19, 21 para ángulos articulares y apertura del gripper (M2, M3) [3, 4]
NumPy	1.24+	BSD	Álgebra vectorial para cálculo angular y manejo de arrays (M3)
Matplotlib	3.7+	PSF	Gráficas de evolución de ángulos articulares y métricas FPS (M5)
scikit-learn	1.3+	BSD	Métricas estadísticas: MAE, correlación de Pearson (evaluación opcional)
Git / GitHub	2.40+	GPL / MIT	Control de versiones y gestión del repositorio del proyecto

Table 6. – Stack de software y librerías del proyecto con versiones y licencias.

Arquitectura General del Pipeline

La figura 1. representa el flujo de datos del sistema desde la captura hasta la visualización.

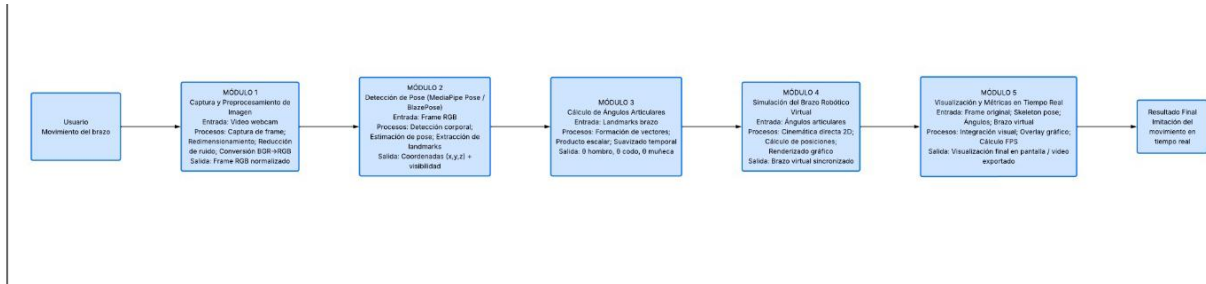


Figure 1. – Arquitectura conceptual del pipeline de procesamiento en tiempo real.

Diseño del Módulo M3 – Cálculo de Ángulos

El módulo M3 implementa el cálculo de ángulos mediante la siguiente lógica para cada articulación:

1. Extraer las coordenadas 2D (x, y) en píxeles de los landmarks proximal, central y distal de la articulación.
2. Construir los vectores $\mathbf{v}_1$ (de central a proximal) y $\mathbf{v}_2$ (de central a distal).
3. Calcular el ángulo mediante la ecuación 2.1:

$$\theta = \arccos\left(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{|\mathbf{v}_1| |\mathbf{v}_2|}\right) \quad (2.1)$$

El resultado en radianes se convierte a grados.

4. Aplicar filtro de media móvil de ventana $w = 5$ sobre el historial de ángulos de la articulación.

5. Retornar el ángulo suavizado junto con un flag de validez (`True` si visibilidad de todos los landmarks > 0.5).
6. Calcular la apertura del gripper: distancia euclidiana entre landmark 19 (punta índice) y landmark 21 (punta pulgar), normalizada entre un valor mínimo d_{min} (pinza cerrada) y máximo d_{max} (pinza abierta), expresada como porcentaje [0%, 100%].

Diseño del Módulo M4 – Cinemática Directa 2D

El brazo robótico virtual se modela como una cadena cinemática de tres eslabones más un efector final tipo gripper, anclada en una posición fija de la pantalla (base). Las longitudes de eslabón son parámetros configurables (en píxeles):

- $L_{hombro} = 120$ px (eslabón 1: base $\rightarrow$ articulación 1)
- $L_{antebrazo} = 100$ px (eslabón 2: articulación 1 $\rightarrow$ articulación 2)
- $L_{mano} = 60$ px (eslabón 3: articulación 2 $\rightarrow$ efector final)
- $L_{gripper} = 30$ px por rama (dos segmentos divergentes que representan la apertura de la pinza; el ángulo entre ellos es proporcional a $a_{gripper}$ calculado en M3)

Las posiciones cartesianas de cada junta se calculan mediante cinemática directa acumulativa (ecuaciones 2.2):

$$\begin{aligned}
 x_1 &= x_0 + L_{hombro} \cos(\theta_{hombro}) \\
 y_1 &= y_0 - L_{hombro} \sin(\theta_{hombro}) \\
 x_2 &= x_1 + L_{antebrazo} \cos(\theta_{hombro} + \theta_{codo}) \\
 y_2 &= y_1 - L_{antebrazo} \sin(\theta_{hombro} + \theta_{codo})
 \end{aligned} \tag{2.2}$$

2.2.4 Bocetos y Flujos de Pantalla

A continuación se describen los bocetos del sistema y los flujos de pantalla principales. Los bocetos representan la disposición visual de la interfaz de usuario en la ventana de visualización de OpenCV.

Boceto 1 – Pantalla Principal (Vista en tiempo real)

La ventana principal muestra el frame de video (640×480 px) con los siguientes elementos superpuestos:

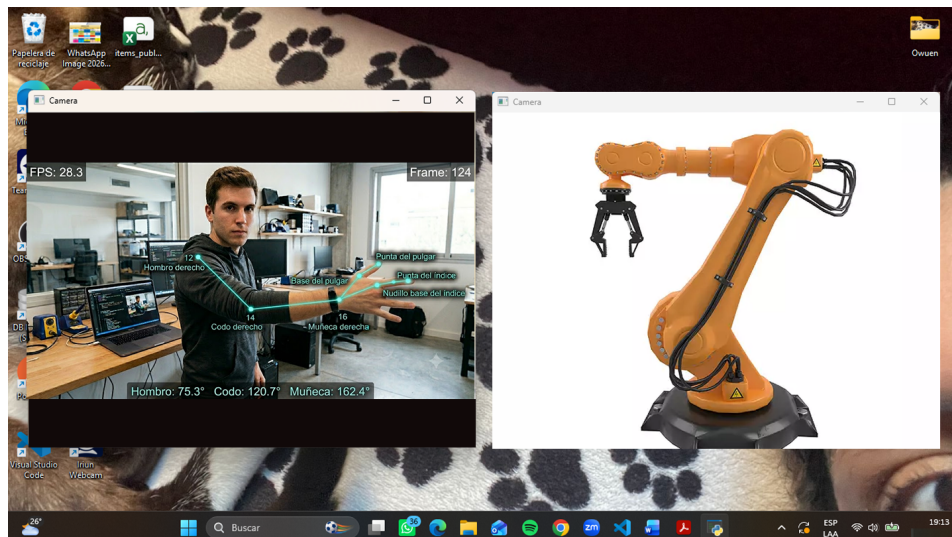


Figure 2. – Boceto de la pantalla principal del sistema (ventana OpenCV en tiempo real). Elementos: contador FPS, número de frame, skeleton MediaPipe con landmarks, brazo robótico virtual, ángulos articulares calculados en tiempo real.

Boceto 2 – Flujo de Pantalla del Sistema

El flujo de interacción del usuario con el sistema sigue la secuencia de estados de la figura 3..

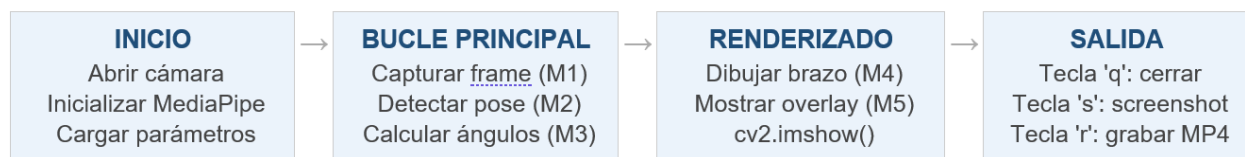


Figure 3. – Flujo de pantalla del sistema

2.3 Plan de Implementación del Proyecto

El plan de implementación sigue la metodología SCRUM con tres sprints. El Sprint #1 (semanas 1–2) cubrió la definición del problema. El Sprint #2 (semanas 3–4) cubre la metodología. El Sprint #3 (fin del curso DIP) abarca el desarrollo completo, análisis de resultados y conclusiones. [2]

Sprint	Semanas	Actividades principales	Entregable
Sprint #1	1–2	Revisión del estado del arte (tema macro, componentes y herramientas). Definición del diagrama de bloques y módulos. Definición de objetivos, entradas y salidas por módulo.	Informe #1 (Secciones 1.1–1.5)
Sprint #2	3–4	Análisis de datos (datasets, características, preprocesamiento). Análisis de confiabilidad y compatibilidad. Definición de requerimientos, recursos, diseño conceptual y bocetos. Plan de implementación.	Informe #2 (Sección 2)
Sprint #3 Fase 1	5–7	M1: captura y preprocesamiento (cv2.VideoCapture, normalización, filtro gaussiano). M2: integración MediaPipe Pose y extracción de landmarks 11–16. M3: cálculo vectorial de ángulos con filtro media móvil y pruebas unitarias. Incluye cálculo de apertura del gripper por distancia euclidiana landmarks 19–21.	Código M1–M3 funcional + pruebas unitarias
Sprint #3 Fase 2	8–10	M4: cinemática directa 2D, renderizado del brazo y simulación visual del gripper (apertura/cierre). M5: visualización integrada y FPS overlay. Integración de M1–M5 en pipeline completo.	Sistema integrado funcional a ≥ 20 FPS
Sprint #3 Fase 3	11–12	Evaluación del sistema: FPS, MAE angular, robustez ante iluminación. Análisis estadístico con scikit-learn y gráficas con matplotlib. Redacción de conclusiones, recomendaciones y futuros trabajos.	Informes #3 y #4 (Secciones 3 y 4)

Table 7. – Plan de implementación del proyecto por sprint y semanas.

2.3.1 Gestión de Riesgos

Se identifican los siguientes riesgos técnicos con sus planes de mitigación:

Riesgo	Probabilidad	Impacto	Mitigación
FPS < 20 en hardware de bajo rendimiento	Media	Alto	Reducir resolución a 320×240 ; desactivar skeleton completo; usar MoveNet como fallback
Oclusión del brazo en poses extremas	Alta	Medio	Implementar umbral de visibilidad (> 0.5); mantener último ángulo válido cuando visibilidad es baja
Ruido excesivo en landmarks por iluminación deficiente	Media	Medio	Aumentar ventana de media móvil a $w = 10$; Inversión de capital en mejorar condiciones de iluminación
Incompatibilidad de versiones de librerías	Baja	Alto	Fijar versiones exactas en requirements.txt; usar entorno virtual Python aislado

Table 8. – Identificación de riesgos técnicos y planes de mitigación.

References

- [1] A. S. B. Pauzi, F. B. M. Nazri, S. Sani, A. M. Bataineh, M. N. Hisyam, M. H. Jaafar, M. N. Ab Wahab, and A. S. A. Mohamed, "Movement estimation using mediapipe blazepose," in *Advances in Visual Informatics. IVIC 2021*, vol. 13051 of *Lecture Notes in Computer Science*, pp. 590–600, Springer, Cham, 2021.
- [2] B. X. Vintimilla, "Propuestas de proyectos – curso dip 2026 pao-i." Diapositivas de presentación, 2026. Escuela Superior Politécnica del Litoral (ESPOL), Facultad de Ingeniería en Electricidad y Computación (FIEC).
- [3] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "Blazepose: On-device real-time body pose tracking," *arXiv*, 2020.
- [4] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "Mediapipe: A framework for building perception pipelines," *arXiv*, 2019.
- [5] G. Bradski, "The opencv library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [6] M. Altayeb, "Hand gestures replicating robot arm based on mediapipe," *Indonesian Journal of Electrical Engineering and Informatics (IJEI)*, vol. 11, pp. 727–737, 09 2023.
- [7] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *Computer Vision – ECCV 2014* (D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, eds.), (Cham), pp. 740–755, Springer International Publishing, 2014.
- [8] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. Pearson, 4th ed., 2018.
- [9] Z. Wang, R. Liang, Z. Chen, and B. Liang, "Fast and intuitive kinematics mapping for human-robot motion imitating: A virtual-joint-based approach," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 10011–10018, 2020.
- [10] C. Xuan, Z. Tang, and J. Xu, "Vision-based robotic arm imitation by human gesture," *arXiv preprint arXiv:1703.04906*, 2017.
- [11] K. Han, S. Li, Z. Li, Y. Zhou, C. Fu, and Y. Xiong, "Robust real-time upper limb motion transfer from human to humanoid for manipulation skill teaching," *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science*, vol. 238, no. 5, pp. 1522–1541, 2024.
- [12] M. Alibeigi, S. Rabiee, and M. N. Ahmadabadi, "Inverse kinematics based human mimicking system using skeletal tracking technology," *Journal of Intelligent & Robotic Systems*, vol. 85, no. 1, pp. 27–45, 2017.